

Convolutional Neural Networks

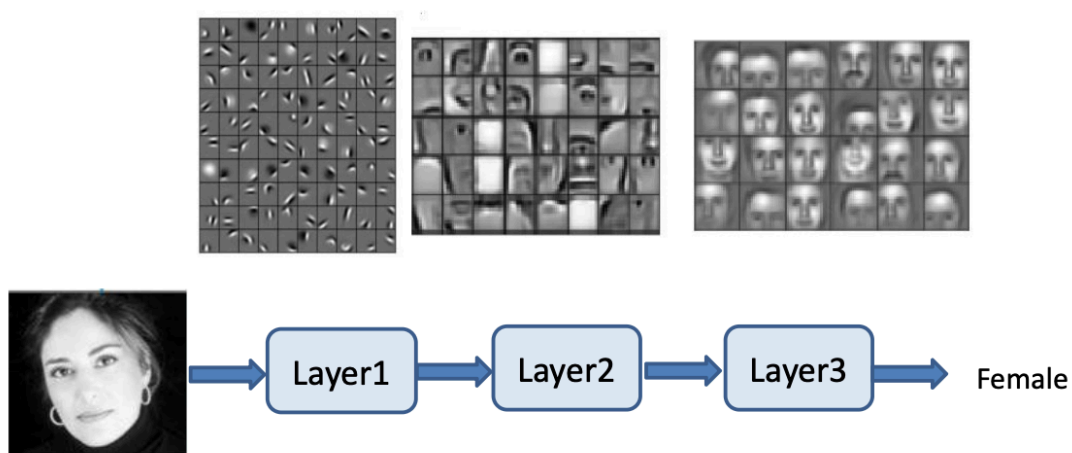
General idea

Convolutional Neural Networks (= CNNs) are **neural networks** used when the **input space** is locally **structured**

- For example in the case of **images** and **languages**
- We need **to keep** such **information**:
 - Traditional Feed-Forward Networks fail as they flatten datapoints in one-dimension vectors
 - CNNs instead **preserve data geometry**

A CNN **learns** a **hierarchy of features**

- **Each layer** extracts **features from** the output of the **previous layer**
- **All layers** are **jointly trained**
- → Learning happens step-by-step
 - **Early layers**: learn **simple patterns**
 - **Mid layers**: learn more **complex structures**
 - **Last layers**: learn the whole **final example**



Convolution

CNNs are Feed-Forward **Networks that use convolution**

in place of matrix multiplication **in at least one of their layers**

Convolution is a **general purpose filter operation for images**

- A **kernel matrix** (= a matrix of weights) is **applied to** an **image**
- The **value** of **each destination** pixel is **computed as** follows:
 - **Moving** the **kernel matrix** over a **central pixel** in the image
 - **Summing** together the **values** of **neighboring pixels**, using the **values** in the **kernel matrix** as **weights**

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n)$$

Where I is the input image, K the kernel matrix

- Allows to smooth, sharpen, enhance, ... specific features of an image
- **! Convolution is a linear operation**
 - **The composition** of convolutions only **would result in a linear function**
 - An activation function is later applied

0	0	0	0	0	0
0	2	3	1	4	0
0	3	1	3	2	0
0	3	0	1	3	0
0	0	2	0	1	0
0	0	0	0	0	0

Input with padding

1	0	-1
1	0	-1
1	0	-1

Kernel matrix

-4	1	-2	4
-4	3	-5	5
-3	2	-3	4
-2	2	-2	1

Output of original size

$$\begin{aligned}
 &2 \cdot 1 + 3 \cdot 0 + 1 \cdot -1 + 3 \cdot 1 + 1 \cdot 0 + 3 \cdot -1 + 3 \cdot 1 + 0 \cdot 0 + 1 \cdot -1 = \\
 &2 + 0 - 1 + 3 + 0 - 3 + 3 + 0 - 1 = \\
 &3
 \end{aligned}$$

CNNs motif: convolution, ReLU, pooling

The layers of a CNN apply **3 sequential steps** (one per layer)

Convolution

Fundamental operation in CNNs

Each filter covers a spatially small portion of the input data → the **receptive field**

- Is **applied in steps** across the dimensions of the input data
- Computes a multi-dimensional feature map

Idea: the **network learns filters** that **activate when** they **see** a specific type of **feature**

ReLU

The **ReLU** activation function is **applied element wise**

$$h(x) = \max(0, x)$$

The step **zeros** all **negative results** of convolution

- **Allows** the **non linearity of the entire architecture** → the **net isn't convolutions only**
- **Doesn't affect** the **receptive layer** of the convolutional field

Idea: allows to learn **complex patterns**

Pooling

Progressively **reduces the spatial size of the representation**

- By **reducing** the **geometric dimension** of matrixes
- Allows to **reduce the amount of parameters**

Idea: allows for **invariance to small spatial translations** and **controls overfitting**